JavaScript Introduction (Detailed Guide for PDF)

What is JavaScript?

JavaScript is a high-level, interpreted programming language used to make web pages interactive and dynamic.

It works with HTML and CSS to build complete web applications.

History of JavaScript

- Created by Brendan Eich in 1995
- Developed at Netscape
- Standardized as ECMAScript (ES)

Why Use JavaScript?

- Adds interactivity to websites
- Handles events (clicks, inputs)
- Used for frontend and backend (Node.js)
- Supports animations and dynamic updates

Features of JavaScript

- Lightweight and fast
- Platform independent
- Object-oriented
- Event-driven
- Supports asynchronous programming

Ways to Use JavaScript

1. Inline JavaScript

```
<button onclick="alert('Hello')">Click</button>
```

2. Internal JavaScript

```
<script>
  alert('Hello World');
</script>
```

3. External JavaScript

```
<script src="script.js"></script>
```

JavaScript Syntax

```
let name = "John";
console.log(name);
```

Variables

```
var x = 10;
let y = 20;
const z = 30;
```

- var → old method
- let → block scoped
- const → constant

Data Types

- String
- Number
- Boolean
- Object
- Array
- Undefined
- Null

Example:

```
let age = 25;
let name = "John";
```

Operators

- Arithmetic (+, -, *, /)
- Comparison (==, ===)
- Logical (&&, ||)

Conditional Statements

```
if (age > 18) {
  console.log("Adult");
} else {
  console.log("Minor");
}
```

Loops

For Loop

```
for (let i = 0; i < 5; i++) {
  console.log(i);
}
```

While Loop

```
let i = 0;
while (i < 5) {
  i++;
}
```

Functions

```
function greet() {
  console.log("Hello");
}

greet();
```

Arrow Functions

```
const add = (a, b) => a + b;
```

Objects

```
let person = {
  name: "John",
  age: 25
};
```

Arrays

```
let numbers = [1, 2, 3, 4];
```

DOM Manipulation

```
document.getElementById("demo").innerHTML = "Hello";
```

Events

```
button.addEventListener("click", function() {
  alert("Clicked!");
});
```

ES6 Features

- let and const
- Arrow functions
- Template literals
- Destructuring

Asynchronous JavaScript

Callback

Promise

Async/Await

Example:

```
async function fetchData() {
```

```
  let response = await fetch("url");
}
```

Error Handling

```
try {
  // code
} catch (error) {
  console.log(error);
}
```

Example Project

HTML:

```
<button onclick="showMessage()">Click Me</button>
<p id="demo"></p>
```

JavaScript:

```
function showMessage() {
  document.getElementById("demo").innerHTML = "Hello JavaScript";
}
```

Advantages of JavaScript

- Fast execution
- Works in all browsers
- Large community

Disadvantages of JavaScript

- Security issues (client-side)
- Browser compatibility differences

Best Practices

- Use meaningful variable names
- Avoid global variables

- Use const and let
- Keep code clean